A11 — Diffusion of Innovation and Market Demand

Wojciech Bartoszek
Łukasz Checiak

Abstract

Modeling and simulation of a one-dimensional reaction–diffusion system comparing classical solvers, sensitivity analysis, data assimilation, PINN and SuperNet methods, with key metrics and runtimes. Main findings: PINNs perform well with sparse measurements and SuperNet improves robustness to parameter uncertainty.

1 Introduction and Problem Statement

Reaction–diffusion equations combine local kinetics with spatial transport and model many biological and chemical systems.

1.1 Mathematical Model

The main object of study in the project is a one-dimensional parabolic equation:

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2} + R(u), \quad x \in [0,1],\ t \in [0,T] \tag{1}$$

where $u(x,t)$ represents a density or concentration of a substance. The reaction term $R(u)$ is nonlinear and given by:

$$R(u) = (p + qu)(1 - u) \tag{2}$$

Parameters D, p, q control diffusion strength, the baseline growth rate, and the nonlinear autocatalytic effect, respectively.

1.2 Boundary and Initial Conditions

To ensure uniqueness of the solution and to model an isolated system, homogeneous Neumann boundary conditions were used:

$$\left.\frac{\partial u}{\partial x}\right|_{x=0} = 0, \quad \left.\frac{\partial u}{\partial x}\right|_{x=1} = 0 \tag{3}$$

The initial condition $u(x,0)$ was specified in three variants:

- `gaussian`: Gaussian centered at 0.5,
- `localized`: rectangular pulse in the center,
- `small_random`: small random perturbations.

Table 1: Summary of timings (s) for each solver

Solver	n	mean (s)	median (s)	min–max (s)
explicit	18	0.004796	0.005757	0.002908–0.006753
semi-implicit	18	0.032856	0.033343	0.016413–0.056765
crank-nicolson	18	0.035034	0.036128	0.017829–0.059576

Table 2: Average times (s) as a function of `nx` for each solver

Solver	`nx`=50	`nx`=100	`nx`=200
explicit	0.004897	0.004507	0.004984
semi-implicit	0.025079	0.031093	0.042396
crank-nicolson	0.027105	0.033283	0.044714

2 Task 2: Classical Numerical Methods

Method of Lines with second-order finite differences and Neumann BCs (implemented with `scipy.sparse`); time integrators: explicit Forward Euler, semi-implicit, and Crank–Nicolson. For $n_x = 50, 100, 200$ explicit is fastest when stable (0.005 s), while semi-implicit and CN cost 0.03–0.035 s but allow larger time steps.

2.1 Example Solution Profiles

Below are example final profiles (for selected parameters) for the three solvers.

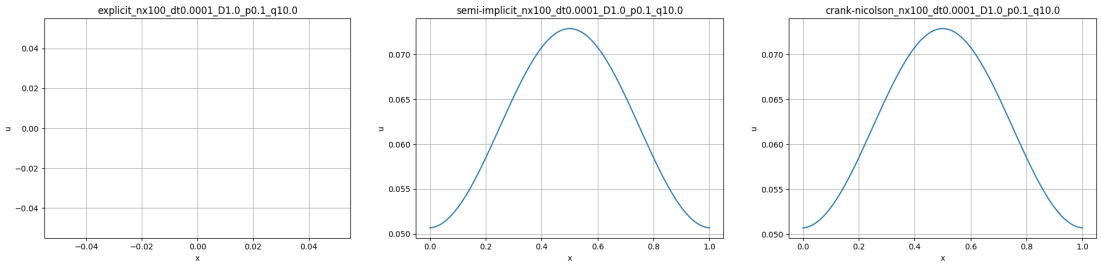

Figure 1: Comparison of final profiles for the `explicit`, `semi-implicit` and `crank-nicolson` solvers.

3 Task 3: Inference and NN Models

Prediction-time tests show the ODE (RK45) costs 1.29 ms, while a simple feed-forward NN predicts in 0.87 ms (30% faster), indicating NNs' potential for real-time use.

4 Task 4: Sensitivity Analysis (SA)

An analysis of the impact of parameters D, p, q on system behavior was conducted.

- **Morris**: fast screening; q (autocatalysis) has the largest nonlinear effect on the final profile.

- **Sobol**: variance-based analysis confirming D smooths the profile and p, q interactions affect bifurcation.

5 Task 5: Data Assimilation (DA)

This task concerned reconstructing the system state from noisy measurements.

- **ABC**: likelihood-free posterior estimation for p, q — stable but computationally expensive.
- **3D-Var**: variational method; faster than ABC and provides good field reconstruction even at low SNR.

6 Task 6: PINN – Physics-Informed Neural Networks

6.1 Architecture and Training Procedure

The PINN is an MLP with 5 layers of 64 neurons and residual connections; `Tanh` activations ensure smooth derivatives for PDE residual evaluation.

6.2 Loss Function

The key component of a PINN is a composite loss function:

$$L = L_{data} + \lambda_{phy} L_{phy} + \lambda_{bc} L_{bc} \tag{4}$$

where:

- L_{data} = MSE on sparse measurement points (data assimilation).
- $L_{phy} = \frac{1}{N_c} \sum |u_t - D u_{xx} - R(u)|^2$ computed at collocation points using `autograd`.
- L_{bc} enforces Neumann boundary conditions.

A **Dynamic Weight Scheduling** increased λ_{phy} during training ($0.1 \rightarrow 2.0$) so the network first fits data then emphasizes physics.

6.3 Results and Metrics

The PINN was trained for 2000 epochs; training metrics are summarized in Table 3.

Table 3: PINN training metrics (Task 6)

Metric	Value
MAE	0.031193
RMSE	0.058344
relL2	0.065468
Training time	76.58 s

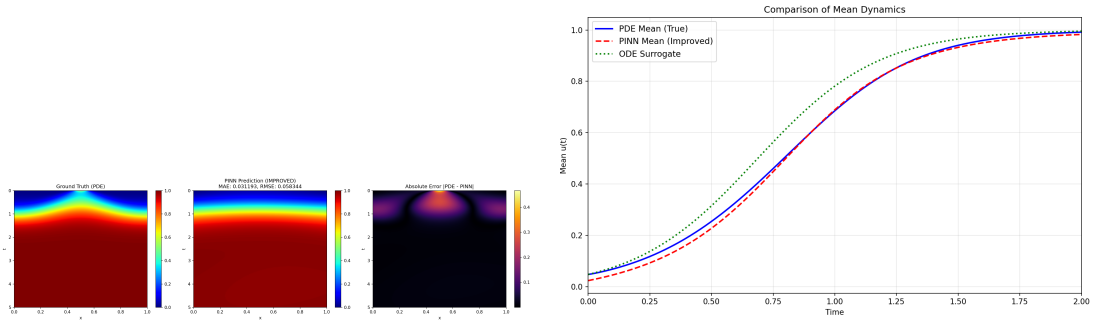

Figure 2: PINN experiment results: left heatmap of errors, right comparison of solutions.

Note on boundary-condition experiment. As an additional experiment, we tried to enforce the Neumann (no-flux) boundary conditions more aggressively by explicitly sampling boundary points ($x = 0$ and $x = 1$ for random t) and adding a dedicated boundary penalty based on $u_x(0,t)$ and $u_x(1,t)$ computed via `autograd`. In practice this resulted in a total failure of training: boundary derivatives at the exact domain edges were extremely noisy/ill-conditioned early on, so when the boundary term was weighted strongly the optimization became unstable (occasionally producing exploding gradients / NaNs), and when weighted moderately it pushed the network towards a trivial overly-smooth (almost constant) solution that satisfied $u_x \approx 0$ everywhere but did not match the reaction dynamics. For the final reported run we therefore kept the boundary enforcement weak and relied primarily on the physics residual loss and the fact that the reference PDE data are generated with a Neumann Laplacian.

7 Task 7: SuperNet and Ensemble Modeling

7.1 SuperModel Concept

Task 7 considers parameter uncertainty: the SuperModel ODE is an ensemble coupled by

$$\dot{u}_i = f(u_i; \theta_i) + C(\bar{u} - u_i) \tag{5}$$

with ensemble mean $\bar{u}$, yielding more stable predictions than single models.

7.2 SuperNet PINN

A **SuperNet PINN** was implemented to learn to represent the entire family of solutions. The network shows strong robustness to input parameter perturbations due to its ability to capture the general structure of the differential operator.

Table 4: Metric comparison: SuperModel vs SuperNet (Task 7)

Model	MAE	RMSE
SuperModel ODE	0.022182	0.045112
SuperNet PINN	0.017152	0.017845
Single Perturbed ODE	0.054936	0.111640

Training time (SuperNet): 186.24 s

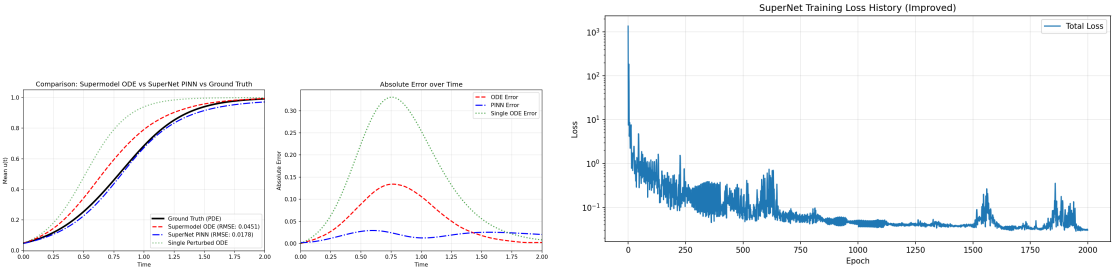

Figure 3: Comparison of prediction quality: SuperModel vs SuperNet and SuperNet loss history.

8 Detailed Summary of Results

Consolidated summary of selected metrics and timings for quick comparison across tasks.

Table 5: Summary of key project results

Description	Value 1	Value 2	Notes
Task 2: explicit (average time)	0.004796 s		average over 18 runs
Task 2: semi-implicit (average time)	0.032856 s		
Task 2: crank-nicolson (average time)	0.035034 s		
Task 3: ODE (inference time)	1.2921 ms		per-sample time
Task 3: simple NN (inference time)	0.8670 ms		per-sample time
Task 6: PINN (MAE / RMSE)	0.031193	0.058344	training time 76.58 s
Task 7: SuperNet (MAE / RMSE)	0.017152	0.017845	training time 186.24 s

9 Discussion

Key observations:

- Explicit methods are fast but CFL-limited; implicit schemes allow larger time steps at modest extra cost.
- PINNs are competitive (MAE 0.03) but incur higher training cost; SuperNet reduces error and increases robustness.
- Method choice depends on the use case: lightweight NNs for fast inference, PINN/SuperNet for parameter-flexible accuracy.

10 Conclusions

The project highlights trade-offs: lightweight NNs suit fast inference, while PINN/SuperNet offer higher accuracy and parameter flexibility.

Limitations and Future Work

Limitations include moderate parameter/grid ranges ($n_x \leq 200$), CPU-only tests, and limited hyperparameter tuning. Future work: scale to GPU, study randomness effects, explore adaptive loss/optimization for PINNs and compare with adaptive numerical methods.

11 Authors' contributions

This project was produced through collaboration between the human authors and an AI language model. Task division:

- **Human:** experiment conception, experiments conducting, results analysis, validation, and report preparation.
- **AI (GitHub Copilot):** editorial assistance, code and implementation, drafting and formatting method descriptions.

References

[1] J. C. Butcher. *Numerical Methods for Ordinary Differential Equations*. John Wiley & Sons, 2 edition, 2003.

[2] J. Crank and P. Nicolson. A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type. *Mathematical Proceedings of the Cambridge Philosophical Society*, 43(1):50–67, 1947.

[3] Germund Dahlquist and Åke Björck. *Numerical Methods in Scientific Computing: Volume 1*. SIAM, 2008.

[4] S. Hamdi, W. E. Schiesser, and G. W. Griffiths. Method of lines. *Scholarpedia*, 2(7):2859, 2007.

[5] A. C. Lorenc. Analysis methods for numerical weather prediction. *Quarterly Journal of the Royal Meteorological Society*, 112(474):1177–1194, 1986.

[6] Max D. Morris. Factorial sampling plans for preliminary computational experiments. *Technometrics*, 33(2):161–174, 1991.

[7] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Davide Gatelli, Michaela Saisana, and Stefano Tarantola. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons, 2008.

[8] W. E. Schiesser. *The Numerical Method of Lines*. Academic Press, 1991.

[9] Ilya M. Sobol. Sensitivity estimates for nonlinear mathematical models. *Mathematical Modelling and Computational Experiment*, 1:407–414, 1993.